

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»**

Кафедра безпеки інформаційних технологій



Основи DevOps та конфігурації локального середовища

МЕТОДИЧНІ ВКАЗІВКИ

до лабораторної роботи № 1

з курсу «Програмування скриптовими мовами»

для студентів спеціальності

«Кібербезпека та захист інформації»

Львів – 2026

Мета роботи: Ознайомитися з концепцією DevOps, принципами управління середовищами розробки, а також отримати практичний досвід зі створення багатофайлових програм з використанням засобів контролю версій.

1. Теоретичні відомості

DevOps (Development and Operations) — це набір практик, що поєднує розробку програмного забезпечення та IT-операції. Основна мета DevOps — скоротити життєвий цикл системи й забезпечити безперервну поставку якісного програмного забезпечення. Важливою складовою DevOps є автоматизація процесів, зокрема через CI/CD (Continuous Integration / Continuous Delivery або Deployment), яка дозволяє часто та надійно випускати оновлення.

DevSecOps (Development, Security, and Operations) — це розширення практик DevOps, яке інтегрує безпеку на всіх етапах життєвого циклу розробки програмного забезпечення. Основна ідея DevSecOps — зробити безпеку спільною відповідальністю всієї команди, а не лише окремого відділу, забезпечуючи автоматизоване тестування безпеки, аналіз залежностей та безперервний моніторинг ще до випуску коду в прод.

SecOps (Security Operations) — це напрям, що поєднує інформаційну безпеку (Security) та IT-операції (Operations) і зосереджується на безперервному моніторингу, виявленні та реагуванні на інциденти безпеки, а також на підтриманні безпечного стану інфраструктури. На відміну від DevSecOps, який інтегрує безпеку безпосередньо в процеси розробки та постачання, SecOps фокусується на операційному захисті середовища в процесі його роботи.

Compliance officer (офіцер із відповідності) — це спеціаліст, відповідальний за дотримання нормативних вимог, стандартів і внутрішніх політик компанії. Він моніторить процеси, оцінює ризики, контролює впровадження безпекових заходів і забезпечує відповідність діяльності чинному законодавству та галузевим регламентам, зокрема в контексті DevOps та циклу розробки програмного забезпечення.

Управління середовищем (Environment Management) передбачає налаштування ізольованих робочих просторів для розробки, тестування та розгортання. Використання віртуальних оточень, пакетних менеджерів (як-от UV) і систем контролю версій (Git) є невід’ємною частиною процесу розробки.

Пакетний менеджер UV (від Astral) встановлюється залежно від операційної системи:

```
# Windows (PowerShell)
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/
install.ps1 | iex"

# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# Або через pip (якщо вже є Python)
pip install uv
```

Після встановлення перевірити версію та створити новий проєкт:

```
uv --version
uv init my-project
cd my-project
uv add requests
uv run python src/my-project/main.py #Файл слід створити
```

Git — це система контролю версій (VCS), що дозволяє відстежувати зміни у файлах, зберігати історію проєкту та спільно працювати над кодом, забезпечуючи можливість створення гілок, злиття змін і відкочування до попередніх версій.

Trunk-Based Development — спрощений та швидкий підхід, у якому всі розробники комітують свій код безпосередньо в єдину головну гілку (main або trunk) або зливають у неї короткоживучі гілки кілька разів на день. Це вимагає високого рівня покриття автотестами та використання прапорів фіч (Feature Flags), але забезпечує максимальну швидкість Continuous Integration/Continuous Delivery (CI/CD).

Git Flow — це суворий, багатогілковий підхід із довготривалими гілками (main для продакшну та develop для розробки), де нові функції створюються в окремих гілках feature/*, а випуск коду в прод відбувається через спеціальні релізні гілки release/* і патчі hotfix/*, що ідеально підходить для проєктів із чіткими плановими релізними циклами та потребою в багаторівневому контролі.

Приклад виконання чекіну репозиторію та додавання двох файлів до нього (Trunk-based підхід — робота безпосередньо в головній гілці main):

```
# Варіант 1: чекін існуючого репозиторію (HTTPS)
git clone https://github.com/<username>/<repo>.git
cd <repo>
```

```
# Варіант 2: ініціалізація локального репозиторію та прив'язка
віддаленого
git init
git remote add origin https://github.com/<username>/<repo>.git

git add lib.py
git commit -m "Add lib module"

git add main.py
git commit -m "Add main entry point"

git push -u origin main
```

Приклад створення feature-гілки (Git Flow підхід), додавання файлу, коміту та відправки змін у віддалений репозиторій:

```
# Створення та перехід у нову feature-гілку
git checkout -b feature/test

# Додаємо новий файл у гілку
git add test.py
git commit -m "Add test module"

# Публікація гілки у віддаленому репозиторії
git push -u origin feature/test
```

2. Завдання

1. Встановити на власному комп'ютері пакетний менеджер **UV** та систему контролю версій **Git**. Також рекомендується обрати та встановити **IDE**.
2. Створити проєкт, що складається з двох файлів, які демонструють механізм імпортування модулів у Python (або іншій мові, узгодженій із викладачем):
 - a. файл **lib.py** — реалізує одну або декілька функцій (довільних);
 - b. файл **main.py** — імпортує потрібні функції з **lib** та викликає їх, виводячи результат.
3. Додати зрозумілі коментарі на рівні методів.
4. Створити порожній проєкт на <https://github.com/> або <https://gitlab.com/> чи іншому довільному репозиторії.
5. Виконати чекін Git-репозиторію (можна з IDE) та додати існуючі файли в кілька комітів використовуючи Trunk-based підхід
6. Створити PR.

7. Створіть довільну (або UML чи C4-діаграму компонентів (Level 3)) яка показує внутрішню структуру Python-скрипту, його методи та зв'язки між ними.
8. Продемонструвати репозиторій та PR.

Не потрібно наводити покрокові інструкції зі встановлення UV чи Git, а також приклади налаштування віддаленого репозиторію на GitHub.

3. Додатково

1. Виконати checkout проекту в окрему папку.
2. Змінити ті ж частини коду в обох папках (спробуйте різні зміни а також ті самі).
3. Створити PR в репозиторії з обох гілок в main. (Git-flow підходом)
4. Продемонструвати як локально такий конфлікт можна виправити (змерджати).

4. Зміст звіту

1. Мета роботи.
2. Повний текст завдання згідно варіанту до кожного пункту.
3. Діаграми що описують розв'язок (C4/UML etc.)
4. Лістинги програм.
5. Результати роботи програм (у текстовій формі).
6. Знімки екрану (фон повинен бути світлим)
7. Висновок.

5. Контрольні запитання

1. Що таке DevOps і які основні цілі він переслідує?
2. Що означає CI/CD і які переваги він надає процесу розробки?
3. Що таке управління середовищем (Environment Management) і навіщо воно потрібне?
4. Які переваги дає використання віртуальних оточень та пакетних менеджерів (наприклад, UV)?
5. Для чого використовується система контролю версій Git?
6. Що таке коміт і яку інформацію він зберігає?
7. Як у Python (або іншій мові) реалізується імпортування модулів між файлами?
8. Які команди Git використовуються для ініціалізації репозиторію та створення коміту?
9. Що таке гілка (branch) у Git і для чого вона використовується?

10. Як створити нову гілку та перемкнутися між гілками?
11. Що таке merge і які конфлікти можуть виникати під час злиття гілок?
12. Що таке trunk-based development і які його основні принципи?
13. Які переваги дає використання гілок у командній розробці?

6. Список літератури

1. Kim, G., Debois, P., Willis, J., Humble, J., & Allspaw, J. (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press.
2. Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. <https://git-scm.com/book>
3. Python Software Foundation. (2026). *The Python Tutorial: Modules*. <https://docs.python.org/3/tutorial/modules.html>
4. Astral. (2026). *uv: An extremely fast Python package and project manager* [Computer software]. GitHub. <https://github.com/astral-sh/uv>

Укладачі:

Д.Л. Гасілін, асистент